Class 31: Good Practices with Docker and CI/CD

Session 31 of 36 – MLOps Block

Data-driven Systems Engineering

Agenda

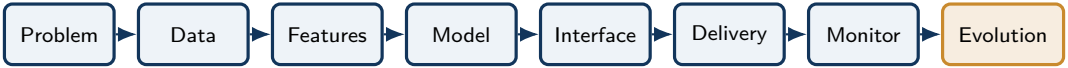
- Why delivery pipelines matter for ML systems.
- Docker and GitHub Actions in the course workflow.
- Pipeline stages, gates, and release safety.
- What to test before deployment.
- Repository examples from the Flask and Streamlit apps.
- Reproducibility, release safety, and rollback thinking.

Why good delivery practice deserves its own class

Learning goals

- Understand how packaging and automation reduce deployment friction.
- Connect notebooks and services to Docker and GitHub Actions.
- Learn what should be tested before a model-backed service is released.
- Explain why CI/CD in ML must validate both software and artifacts.

Course Map and Running Cases



Today in the course

Focus: Packaging, testing, and automated delivery.
Bridge: This class moves the course from local development into repeatable release processes that reduce deployment risk.

Fraud case

In the fraud case, the scoring service becomes containerized, testable, and suitable for CI-driven release checks.

Pasteurization case

In the pasteurization case, dashboards and streaming services gain a repeatable environment so demonstrations do not depend on one laptop state.

Course thread: The key course shift is from building a system to proving that it can be rebuilt and shipped consistently.

A delivery pipeline is a risk-control mechanism

- CI/CD is not only about speed. Its main purpose is to make releases safer and more repeatable.
- In ML systems, the release unit often includes code, dependencies, configuration, and trained artifacts.
- A pipeline reduces the number of assumptions hidden inside one developer machine.

From local success to repeatable delivery



- Deterministic installs and smoke tests should happen before container release.
- Deployment is not the end; health checks and logs are part of the pipeline.

Why ML delivery is harder than standard app delivery

- The build may depend on large artifacts that are not regenerated on every commit.
- A working container can still be wrong if the embedded model or scaler is stale.
- Validation must check not only whether the service starts, but whether it predicts correctly on representative inputs.
- Delivery failures may appear as silent behavior changes, not only as crashes.

Repo materials we should exploit

Repo activity

- ‘tutorials/Docker/tutorial.md‘, ‘node.Dockerfile‘, and ‘docker-compose.yaml‘
- ‘tutorials/GitHubActions/tutorial.md‘

These are already present, so the lecture should tie them directly to the Flask and Streamlit apps.

Role of the main technologies

Docker	Builds a portable runtime for the app and its dependencies.
Docker Compose	Coordinates several containers, such as a service, an API, and a database, as one stack.
GitHub Actions	Runs repository-native workflows for testing, building, or deployment steps.
Git	Triggers the history and events around which automation can run.

Note

Jenkins remains available in ‘tutorials/Jenkins‘ for teams that want a self-hosted alternative, but this class focuses on Docker and GitHub Actions.

Typical stages in a small ML delivery pipeline

1. Check out code and artifacts.
2. Install dependencies in a deterministic environment.
3. Run unit or smoke tests on preprocessing and inference.
4. Build the container image.
5. Run container-level checks such as startup and health endpoints.
6. Publish or deploy only if the gates pass.

Dockerfile layering, annotated in the repo

```
FROM node:18-alpine
WORKDIR /app
COPY package*.json ./
RUN npm ci --only=production
COPY . .
RUN adduser -D appuser && chown -R appuser /app
USER appuser
EXPOSE 3000
HEALTHCHECK --interval=30s --timeout=3s --retries=3 \
  CMD curl -f http://localhost:3000/health || exit 1
```

- ‘tutorials/Docker/node.Dockerfile‘ orders layers so dependency installs are cached separately from source code.
- It also shows two good practices worth carrying into a Python service: a non-root user and a HEALTHCHECK.

Composing services with Docker Compose

```
services:
  api:
    build: ./api
    environment:
      DATABASE_URL: postgres://user:pass@db:5432/app
    depends_on:
      - db
  db:
    image: postgres:15-alpine
    volumes:
      - pgdata:/var/lib/postgresql/data
```

- ‘tutorials/Docker/docker-compose.yaml‘ shows a small multi-service stack with explicit dependency ordering.
- A fraud API plus a feature store, or a pasteurization stream plus a dashboard, follow the same pattern.

GitHub Actions example from the tutorial

```yaml
on:
  push:
  schedule:
    - cron: "0 2 * * *"

jobs:
  test:
    runs-on: ubuntu-latest
```

- ‘tutorials/GitHubActions/tutorial.md‘ is useful because it shows that workflows are event-driven.
- For this course, the key idea is that pushes and scheduled checks can both guard the service.

What to test in ML delivery

- Schema validation for inputs and outputs.
- Serialization compatibility for model artifacts.
- Inference latency under small load.
- Reproducibility of a baseline metric on a frozen sample.
- Container startup and health endpoint behavior.

Release gates for model-backed services

- Does the artifact load with the current runtime and library versions?
- Does a frozen inference sample still produce acceptable output?
- Does the container expose the required route and health response?
- Are secrets, environment variables, and external dependencies configured correctly?

Rollback and observability are part of delivery

- Safe delivery includes a way to revert a bad release quickly.
- Observability tells us whether a release is healthy after deployment.
- Logs, metrics, and health endpoints are part of the delivery design, not a postscript.

Course connection

This is the bridge from software engineering into operational MLOps: the deployment pipeline and the monitoring pipeline eventually meet in Class 32.

Discussion prompt

Question

A container builds, starts, and answers health checks, but it silently serves a stale model. Did the pipeline do its job?

- This separates “the service runs” from “the service is correct.”
- It motivates testing frozen inference samples, not only container startup.

Papers and materials

Papers and materials

Docker get started; Docker CLI reference; GitHub Actions documentation; Continuous Delivery for Machine Learning.

From This Class to the Next

What stays with us

- Delivery is a repeatable engineering process, not a one-time handoff.
- Containers and CI/CD reduce environment drift and manual release mistakes.
- Release quality depends on tests, health checks, rollback paths, and observability.

Project use

Teams can now describe how code, dependencies, models, and checks travel from the repository into a running service.

Next connection

Class 32 continues past the release moment into continuous monitoring and scalability: what happens to this pipeline's output once it carries real load.

Course thread: Reliable delivery matters most when the team also knows how the shipped system behaves under sustained use.